Seguridad Informática

Trabajo Práctico 2

CryptoFile: cifrado de archivos con AES-256



Integrantes:

- Abregú, Santiago
- Futryk, Fabricio
- Viazzo, Lucía

2do cuatrimestre 2026

Resumen **3**
1. Funcionamiento del código 3
Estructura 3
2. Decisiones de diseño 4
Modo de operación: AES-256-GCM 4
Derivación de la clave: Argon2id 4
IV / nonce 4
Formato del archivo cifrado (.cryf) 4
Errores 5
3. Pruebas 5
4. Anexo de ejecución 6
Requisitos 6
Instalación 6
Línea de comandos 7
Interfaz web 7
Pruebas automáticas 7

Resumen

CryptoFile cifra cualquier archivo utilizando AES-256-GCM. La clave se deriva de una contraseña con Argon2id y una sal aleatoria, y el resultado se descifra a un archivo idéntico byte a byte. Se usa desde la línea de comandos (`cli.py`) o desde una interfaz web hecha con Streamlit (`app.py`). Las dos utilizan el mismo mecanismo de criptografía apoyado en la librería `cryptography` de Python.

Un deploy de la versión web de CryptoFile se puede encontrar en la siguiente URL: https://cryptofile.streamlit.app/

1. Funcionamiento del código

Estructura

ARCHIVO	RESPONSABILIDAD
`cryptofile/core.py`	Formato del archivo cifrado, derivación de clave, `encrypt()` y `decrypt()`, errores (`AuthenticationError`, `InvalidFormatError`).
`cli.py`	Comandos `encrypt`, `decrypt` y `sha256`.
`app.py, i18n.py`	Interfaz web (Streamlit). Procesa todo en memoria y ofrece el resultado como descarga.
`tests/`	Pruebas automáticas con `pytest`: ida y vuelta, contraseña incorrecta, archivo alterado, sal/IV distintos y nombres de archivo maliciosos.

Flujo de cifrado

1. Leer el archivo y la contraseña.
2. Generar con `os.urandom` una sal de 16 bytes y un nonce de 12 bytes.
3. Derivar la clave de 32 bytes: `Argon2id(contraseña, sal, t=3, m=64 MiB, p=4)`.
4. Armar el encabezado de 43 bytes (magic, versión, parámetros de Argon2id, sal y nonce).
5. Cifrar `nombre original + contenido` con AES-256-GCM. El encabezado se pasa como dato asociado (AAD).
6. Escribir `encabezado ‖ texto cifrado ‖ tag (16 bytes)`.

Flujo de descifrado

1. Leer el encabezado y validarlo: magic, versión y parámetros dentro de rango.
2. Re-derivar la clave con la contraseña ingresada y la sal y los parámetros guardados.
3. Descifrar con AES-256-GCM, que verifica el tag sobre el encabezado y el texto cifrado.
4. Si el tag no coincide, informar el error y no escribir nada.
5. Si coincide, recuperar el nombre original y escribir el archivo descifrado.

2. Decisiones de diseño

Modo de operación: AES-256-GCM

Elegimos GCM y no CBC+HMAC porque es un modo de cifrado autenticado (AEAD): en una sola operación y con una sola clave da confidencialidad e integridad, y eso es lo que permite detectar si el archivo fue modificado. El encabezado se pasa como dato asociado (AAD): no se cifra, pero queda autenticado, así que alterar cualquier parámetro, la sal o el nonce también invalida el tag. Con CBC+HMAC habría que derivar dos claves, aplicar padding y respetar el orden encrypt-then-MAC, y cada uno de esos pasos es un punto más donde se puede cometer un error.
No elegimos ECB porque cifra de igual manera los bloques iguales, dejando ver así patrones dentro del archivo.

Derivación de la clave: Argon2id

La contraseña nunca se usa directamente como clave: pasa por Argon2id, la opción recomendada por la RFC 9106 y por OWASP. A diferencia de PBKDF2, Argon2id es memory-hard: cada intento obliga a reservar 64 MiB de memoria, lo que encarece mucho un ataque de fuerza bruta con GPU o hardware dedicado. Usamos t=3, m=64 MiB y p=4, una de las configuraciones recomendadas por la RFC 9106: tarda una fracción de segundo por operación y entra en la memoria de un servidor gratuito. La sal es aleatoria y distinta en cada cifrado: dos archivos con la misma contraseña quedan con claves diferentes y no sirven *rainbow tables*. Los parámetros se guardan en el encabezado para poder cambiarlos en el futuro sin perder la capacidad de descifrar archivos viejos.

IV / nonce

El nonce de GCM es de 96 bits, el tamaño estándar, y se genera aleatoriamente en cada cifrado. En GCM es crítico no repetir nunca un par (clave, nonce). Acá además cada cifrado usa una clave distinta, porque la sal es nueva, así que una repetición es prácticamente imposible.

Formato del archivo cifrado (`.cryf`)

BYTES	CAMPO	EJEMPLO REAL (DOCUMENTO_SECRETO.PDF.CRYF)
0–3	Magic `CRYF`	`43 52 59 46`
4	Versión del formato	`01`
5	KDF (1 = Argon2id)	`01`
6–9	Iteraciones (t)	`00 00 00 03`
10–13	Memoria en KiB (m)	`00 01 00 00  (65 536 KiB = 64 MiB)`
14	Paralelismo (p)	`04`
15–30	Sal (16 bytes)	`6c 13 4e 8b 56 c5 b6 f3 ef 4d 95 44 35 c2 0a 83`
31–42	Nonce (12 bytes)	`98 78 ec ef cd e6 d9 50 37 bf 0e 96`
43–…	Texto cifrado + tag	`1791 bytes = 2 + 21 (nombre) + 1752 (contenido) + 16 (tag)`

El archivo trae todo lo necesario para descifrarlo salvo la contraseña. También cifra el nombre original, así el descifrado lo restaura y el nombre no queda expuesto. El archivo cifrado ocupa 43 + 2 + largo del nombre + 16 bytes más que el original.

Errores

GCM no puede distinguir una contraseña incorrecta de un archivo modificado: en los dos casos el tag no verifica. Por eso el programa informa un único error, "la contraseña es incorrecta o el archivo fue modificado", sin generar una salida: el descifrado se hace completo en memoria y el archivo se escribe solo si el tag verifica, así que nunca se entrega contenido sin autenticar. Los archivos que no tienen el formato de CryptoFile, que están truncados o que tienen parámetros fuera de rango se rechazan antes de derivar la clave.

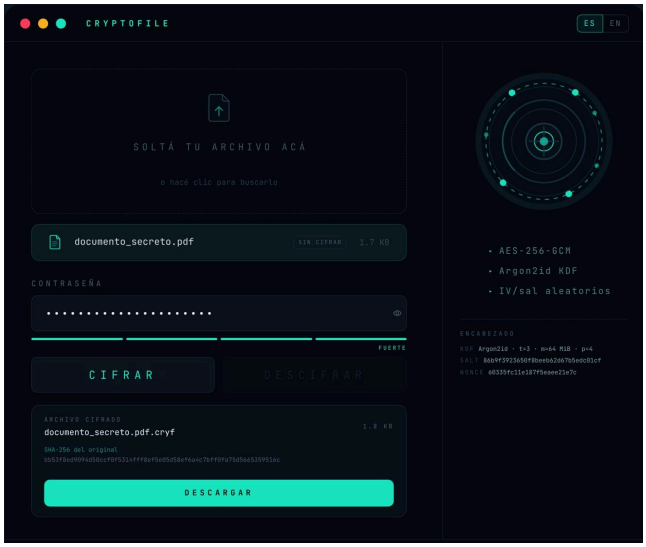
3. Pruebas

Ciclo completo con un PDF, comparando el SHA-256 del original con el del descifrado. También se prueba una contraseña incorrecta y un archivo con un solo bit modificado:

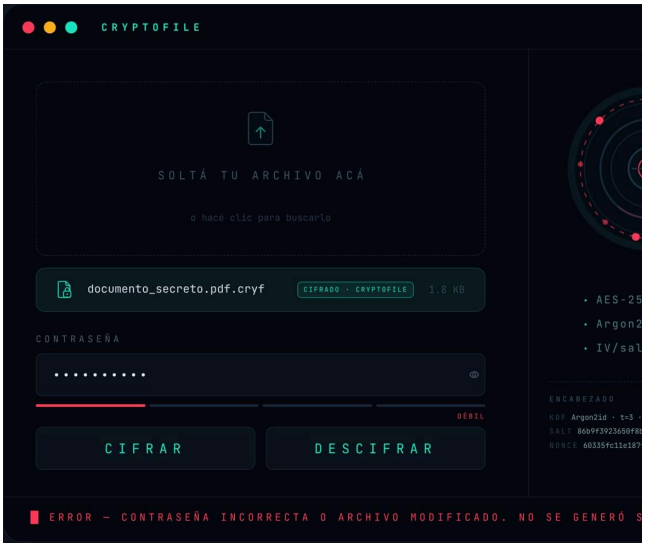
```
$ uv run cli.py encrypt documento_secreto.pdf
Cifrado OK -> documento_secreto.pdf.cryf (1834 bytes)
$ uv run cli.py decrypt documento_secreto.pdf.cryf -o documento_descifrado.pdf
Descifrado OK -> documento_descifrado.pdf (1752 bytes)
$ uv run cli.py sha256 documento_secreto.pdf documento_descifrado.pdf
bb53f8ed9094d50ccf0f5314fff8ef5e05d58ef6a4c7bff0fa75d5665359516c  documento_secreto.pdf
bb53f8ed9094d50ccf0f5314fff8ef5e05d58ef6a4c7bff0fa75d5665359516c  documento_descifrado.pdf
Los hashes COINCIDEN.
$ uv run cli.py decrypt documento_secreto.pdf.cryf -o x.pdf   # contraseña incorrecta
Error: No se pudo descifrar: la contraseña es incorrecta o el archivo fue modificado.
(código de salida 1 — x.pdf no se creó)
$ uv run cli.py decrypt alterado.cryf -o y.pdf   # 1 bit modificado en el byte 100
Error: No se pudo descifrar: la contraseña es incorrecta o el archivo fue modificado.
(código de salida 1 — y.pdf no se creó)
```

Los hashes coinciden, por lo tanto, el archivo descifrado es idéntico al original. Las pruebas automáticas (uv run pytest) verifican además:

- ida y vuelta byte a byte con archivos de 0, 1, 15, 16, 17, 1024 y ~1 MB;
- que dos cifrados del mismo archivo con la misma contraseña usen sal y nonce distintos;
- que se detecte la modificación de cualquier parte del archivo (parámetros, sal, nonce, texto cifrado o tag);
- que la CLI no cree el archivo de salida ante contraseña incorrecta o archivo alterado;
- que el nombre de archivo guardado en el .cryf no permita escribir fuera de la carpeta elegida.



Interfaz web después de cifrar: SHA-256 del original y parámetros del encabezado.



Descifrado con contraseña incorrecta: se informa el error y no se ofrece salida.

4. Anexo de ejecución

Requisitos

uv (recomendado) o Python 3.11 o superior con pip. Dependencias (pyproject.toml / requirements.txt): `cryptography ≥ 44`, que incluye Argon2id, y `streamlit`, solo para la interfaz web.

Instalación

Con uv no hace falta instalar nada a mano: `uv run` lee `pyproject.toml` y `uv.lock`, y crea el entorno virtual con las versiones exactas la primera vez:

```
uv run streamlit run app.py
uv run cli.py encrypt documento.pdf
uv run pytest
```

Alternativa con pip. Con el entorno virtual activado, los comandos de este anexo se usan igual cambiando `uv run` por `python` (por ejemplo, `python cli.py encrypt documento.pdf`). Las excepciones son la interfaz web, que se levanta con `streamlit run app.py`, y las pruebas, que se corren con `python -m pytest`. Para instalar:

```
# Windows (PowerShell)
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt

# Linux / macOS
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Línea de comandos

```
# Cifrar (pide la contraseña dos veces, sin mostrarla) → genera documento.pdf.cryf
uv run cli.py encrypt documento.pdf

# Descifrar → restaura el nombre original, o el indicado con -o
uv run cli.py decrypt documento.pdf.cryf -o documento_descifrado.pdf

# Comparar hashes
uv run cli.py sha256 documento.pdf documento_descifrado.pdf

# Ayuda
uv run cli.py --help
```

Interfaz web

```
uv run streamlit run app.py   # abre http://localhost:8501
```

Arrastrar un archivo a la zona de carga, escribir la contraseña (Enter para aplicarla) y presionar CIFRAR o DESCIFRAR. El botón de descifrar solo se habilita si el archivo tiene el formato de CryptoFile. El resultado se baja con DESCARGAR, junto con su SHA-256. El selector ES/EN cambia el idioma.

Pruebas automáticas

```
uv run pytest
# con pip: pip install -r requirements-dev.txt y python -m pytest
```